INSTITUTO DE FORMACIÓN TÉCNICA SUPERIOR N° 29 (IFTS 29)
TECNICATURA SUPERIOR EN DESARROLLO DE SOFTWARE

PROGRAMACIÓN SOBRE REDES

Práctica Formativa Obligatoria N° 3 (PFO 3)

Rediseño como Sistema Distribuido
(Cliente-Servidor Concurrente con Sockets TCP y Worker Pool)

Docente:	Prof. Alan Portillo
Alumno:	Guillermo Sciulli
Materia:	Programación sobre Redes – COM. E
Repositorio GitHub:	https://github.com/guillote1977/ProgSRedes-PFO3
Estado de Entrega:	Completo (Código funcional + Diagrama + Documentación)

1. Enlace al Repositorio y Datos de Entrega

El presente documento formaliza la entrega de la Práctica Formativa Obligatoria N° 3 (PFO 3). Todo el código fuente, la implementación en Python, el diagrama de arquitectura en formatos raster y vectorial, y los ejemplos de prueba se encuentran versionados en el siguiente repositorio oficial de GitHub:

> **ENLACE OFICIAL AL REPOSITORIO GITHUB:** https://github.com/guillote1977/ProgSRedes-PFO3
> El repositorio es público, permitiendo la clonación directa.

2. Objetivos y Consignas del Trabajo

Objetivo Principal: Transformar un sistema tradicional en una arquitectura distribuida escalable y desacoplada utilizando sockets y componentes modernos de infraestructura para alta disponibilidad.

Consignas requeridas y cumplimentadas:

- **Diseño de Arquitectura Distribuida:** Diagrama integral que modela clientes móviles/web, balanceador de carga (Nginx/HAProxy), servidores workers con pool de hilos, cola de mensajes (RabbitMQ) y almacenamiento distribuido (PostgreSQL y S3).

- **Implementación del Servidor Socket en Python:** Servidor multi-hilo no bloqueante que atiende clientes en hilos independientes y delega el procesamiento de cómputo a un ThreadPoolExecutor (Worker Pool).

- **Implementación del Cliente Socket en Python:** Cliente interactivo con menú en consola para envío de tareas síncronas/asíncronas, consulta diferida de estado y ráfagas de prueba de carga concurrente.

- **Entregables:** Diagrama de arquitectura del sistema distribuido y código fuente completo en repositorio GitHub.

3. Diagrama de Arquitectura del Sistema Distribuido

Se diseñó una topología distribuida de cinco niveles que desacopla la recepción de conexiones, el enrutamiento de red, el cómputo en paralelo y la persistencia de datos:

Capa / Componente	Tecnología Adoptada	Función en la Arquitectura
1. Clientes Heterogéneos	Aplicaciones Móviles y Web	Generan solicitudes mediante conexiones TCP directas o peticiones REST/WebSocket hacia la plataforma.

2. Balanceador de Carga	Nginx / HAProxy	Punto único de ingreso público. Distribuye las cargas (Round-Robin / Least Connections) y previene la caída total del sistema (anti-SPOF).
3. Servidores Workers	Python + ThreadPoolExecutor	Nodos de cómputo distribuidos. Cada servidor maneja su pool de hilos para ejecutar tareas en paralelo sin bloquear la recepción.
4. Cola de Mensajes	RabbitMQ (AMQP)	Middleware asíncrono que desacopla la recepción del procesamiento y amortigua picos extremos de tráfico sin pérdida de datos.
5. Almacenamiento Distribuido	PostgreSQL + AWS S3 / MinIO	Persistencia híbrida: base de datos relacional ACID para estados y transacciones; almacenamiento de objetos para archivos pesados.

4. Implementación del Software (Sockets TCP en Python)

La solución de software desarrollada cumple estrictamente con el enfoque cliente-servidor distribuido utilizando la biblioteca estándar de Python sin requerir dependencias externas de terceros.

A. Servidor Concurrente con Worker Pool (`servidor.py`)

El servidor (`servidor.py`) implementa un diseño de doble concurrencia:

1. Concurrencia a nivel de red: Cada cliente conectado se delega a un hilo daemon (`threading.Thread`) exclusivo, garantizando que múltiples clientes interactúen de forma simultánea.

2. Concurrencia a nivel de cómputo: Las tareas solicitadas se despachan a un `ThreadPoolExecutor(max_workers=4)`, emulando el cluster de workers del diagrama arquitectónico.

3. Almacenamiento thread-safe: Se utiliza un lock (`threading.Lock`) para proteger el diccionario global de estados de tareas, asignando a cada solicitud un identificador único con timestamp (`TASK-<milisegundos>-<*UUID*>`), garantizando colisión cero ante ráfagas de solicitudes concurrentes.

B. Protocolo de Mensajería JSON sobre TCP

La comunicación se estructura mediante tramas serializadas en formato JSON estructurado:

Comando	Payload Enviado por Cliente	Respuesta del Servidor
ENVIAR_TAREA	`{"tipo": "calcular_cuadrado", "payload": {"numero": 12}}`	`{"status": "OK", "task_id": "TASK-1793...", "mensaje": "Tarea recibida"}`
CONSULTAR_TAREA	`{"task_id": "TASK-1793..."}`	`{"status": "OK", "info": {"estado": "COMPLETADA", "resultado": 144}}`
SALIR	`{"comando": "SALIR"}`	`{"status": "OK", "mensaje": "Conexión finalizada"}`

C. Cliente Interactivo y Pruebas de Carga (`cliente.py`)

El cliente (`cliente.py`) provee un menú interactivo en consola que permite:
• Enviar tareas matemáticas (`calcular_cuadrado`) y de procesamiento de cadenas (`procesar_texto`).
• Consultar en cualquier momento el estado y resultado de una tarea mediante su ID único.
• Ejecutar una prueba de carga concurrente enviando 3 tareas simultáneas para verificar el despacho dinámico en los 4 hilos del Worker Pool.

5. Guía de Ejecución y Validación de Pruebas

Para ejecutar y probar la solución en cualquier entorno con Python 3 instalado:

Paso 1: Clonar el Repositorio de GitHub

```
git clone https://github.com/guillote1977/ProgSRedes-PFO3.git
cd ProgSRedes-PFO3
```

Paso 2: Iniciar el Servidor Distribuido

```
python3 servidor.py
(Escuchando en TCP 127.0.0.1:5000 con 4 workers activos)
```

Paso 3: Iniciar el Cliente en otra terminal

```
python3 cliente.py
(Aparecerá el menú interactivo para despachar tareas y consultar estados)
```